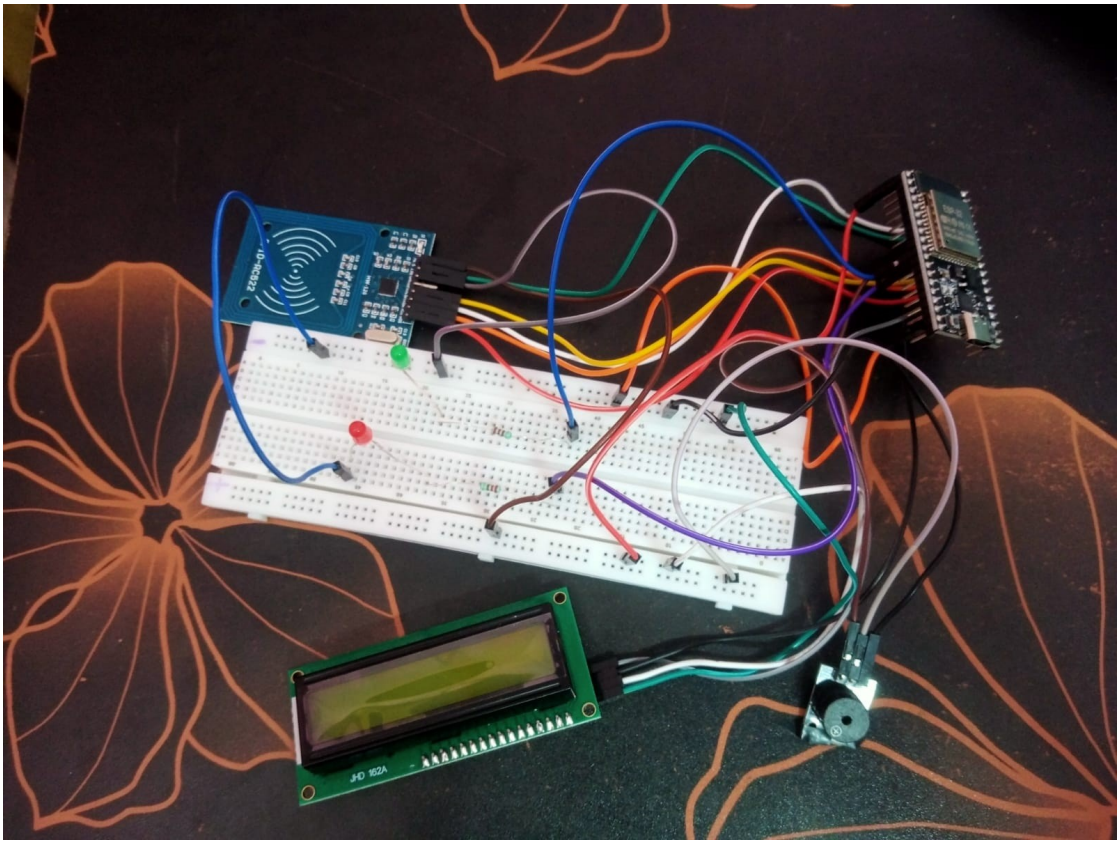
MAJOR PROJECT REPORT

SMART ATTENDANCE SYSTEM USING RFID

ESP32 • RC522 RFID • 16x2 I2C LCD • LED and buzzer feedback



PROJECT DOCUMENTATION

Abstract

The Smart Attendance System Using RFID automates attendance marking using an ESP32 microcontroller and an RC522 RFID reader. Each RFID card has a unique identifier (UID), which is checked against registered student records. A 16x2 I2C LCD displays prompts and attendance status, while green and red LEDs and a buzzer provide feedback. Student UID-to-name mappings are stored in ESP32 Preferences, and attendance marks are tracked during the current powered session. Attendance events and elapsed uptime are available through the Serial Monitor. The completed prototype demonstrates a practical RFID-based attendance workflow.

Table of Contents

- 1. Introduction
- 2. Problem Statement
- 3. Aim and Objectives
- 4. Scope and Proposed Solution
- 5. Components Required
- 6. System Architecture
- 7. Circuit Design and Pin Mapping
- 8. Working Principle and Algorithm
- 9. Software Implementation
- 10. Testing and Results
- 11. Project Photos and Evidence
- 12. Limitations and Future Enhancements
- 13. Conclusion
- Appendix A - Complete Arduino Source Code

1. Introduction

Attendance is commonly recorded manually or through separate administrative tools. Manual entry takes time and can introduce mistakes. RFID (Radio Frequency Identification) allows a reader to communicate with a compatible tag and retrieve its identifier without typing a student name for every attendance entry.

This project uses an ESP32 as the controller, an RC522 reader for card detection, and a 16x2 LCD for local user feedback. LEDs and a buzzer give additional indication. The project is designed as a small classroom or lab prototype and can be extended later with real-time clock or network-based reporting.

2. Problem Statement

Manual attendance can be time-consuming, inconsistent, and vulnerable to recording mistakes. A simple RFID-based prototype can streamline card identification and provide immediate status feedback. The system still requires sensible registration procedures and secure handling of attendance records; RFID alone does not guarantee a person's identity or prevent card sharing.

3. Aim and Objectives

Aim: To design a microcontroller-based attendance prototype that reads RFID cards, checks whether their UIDs are registered, and provides status feedback.

- Interface an RC522 RFID reader with the ESP32 over SPI.
- Display the system prompt and scan results on a 16x2 I2C LCD.
- Register a UID with a student name using Serial Monitor commands.
- Store registered student details in ESP32 Preferences so they can persist after reboot.
- Indicate successful, duplicate, and unregistered scans using LCD text, LEDs, and buzzer feedback.
- Print attendance events and elapsed uptime information in the Serial Monitor.

4. Scope and Proposed Solution

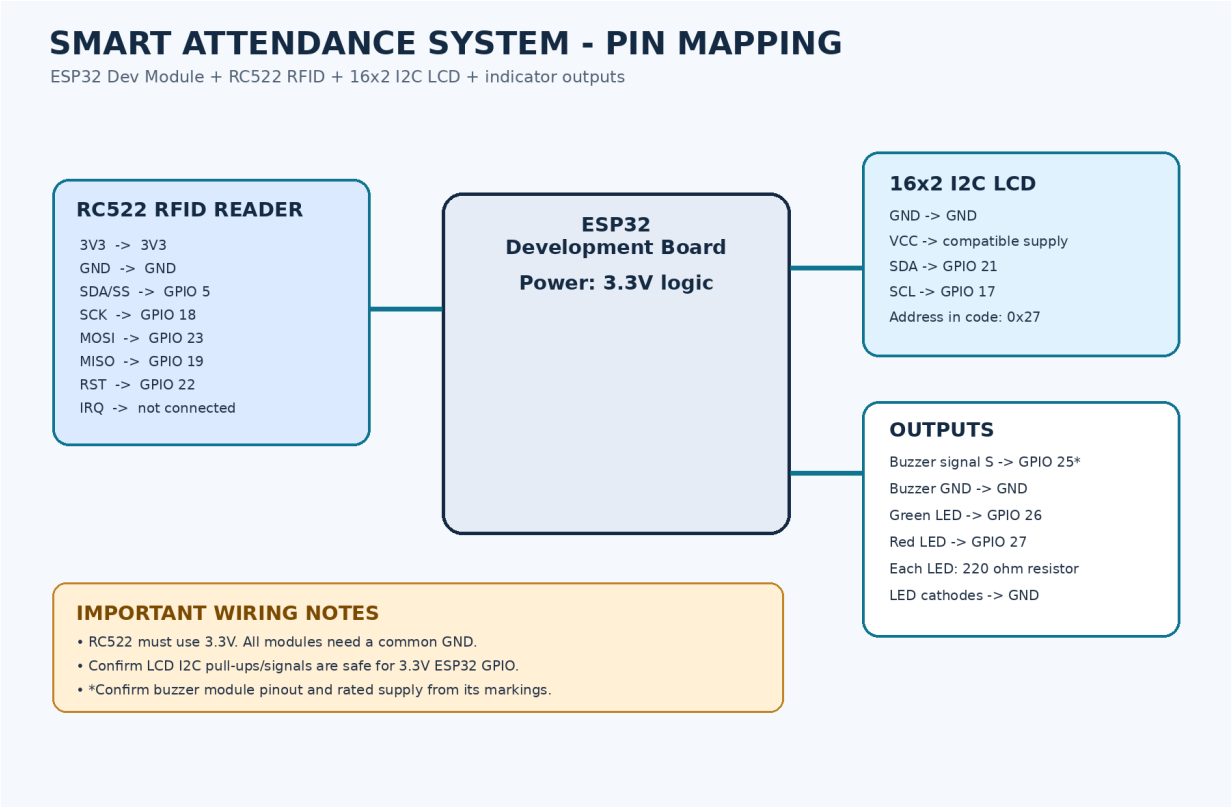
The current source code supports up to 30 registered students. A card UID must be registered with a name before it is recognized as a valid attendance entry. Each registered UID is marked once during the current power session; another scan in that session produces an “Already marked” message. Registered students persist through ESP32 Preferences, but attendance event history is not stored persistently by this code. No SD-card database, cloud database, RTC module, or network-time synchronization is implemented.

5. Components Required

Component	Purpose / notes
ESP32 development board	Main controller; runs the RFID, display and indicator logic.
RC522 RFID reader and compatible card/tag	Reads the card UID over SPI; use 3.3V logic/power.
16x2 LCD with I2C adapter	Shows startup prompt and scan status.
Green LED + 220 ohm resistor	Visual indicator for accepted/previously attended card.
Red LED + 220 ohm resistor	Visual indicator for unregistered card.
3-pin buzzer module	Provides audible feedback for accepted, unregistered, and repeated card scans.
Breadboard, jumper wires, USB cable	Prototyping, interconnection and programming.

6. System Architecture

Input: RFID card/tag -> RC522 reader -> ESP32 UID-processing logic. The controller checks the UID against the registered student list stored in Preferences, then checks whether that UID has already been marked during the current power session.



Outputs: LCD status, red/green indicator LEDs, buzzer feedback, and Serial Monitor messages. The pin-mapping drawing shows the configuration used for the tested prototype.

7. Circuit Design and Pin Mapping

Module pin / signal	ESP32 connection
RC522 SDA / SS	GPIO 5
RC522 SCK	GPIO 18
RC522 MOSI	GPIO 23
RC522 MISO	GPIO 19
RC522 RST	GPIO 22
RC522 3.3V and GND	3V3 and common GND; do not power RC522 from 5V
LCD SDA / SCL	GPIO 21 / GPIO 17
LCD I2C address in code	0x27 (some adapters may use 0x3F)
Buzzer signal S	GPIO 25, only after confirming module pinout
Green LED	GPIO 26 through 220 ohm resistor; LED cathode to GND
Red LED	GPIO 27 through 220 ohm resistor; LED cathode to GND

Electrical caution: ESP32 GPIOs are not 5V tolerant. If an LCD I2C backpack is powered from 5V and pulls SDA/SCL up to 5V, use a suitable bidirectional level shifter or another verified 3.3V-safe arrangement. Do not connect unknown-voltage signals directly to ESP32 pins. Confirm the buzzer module's rated supply and pin labels.

8. Working Principle and Algorithm

1. On startup, initialize Serial Monitor, GPIO outputs, I2C LCD, SPI bus, and RC522 reader.
2. Load stored student UID/name records from ESP32 Preferences.
3. Show the home prompt: “Smart Attendance / Scan your card”.
4. When a card is detected, read and normalize the UID.
5. If the UID is not registered, show “Card not found / Not registered”, switch on the red LED, and give error beeps.
6. If the UID is registered but already marked in the current session, show “Already marked” and give feedback.
7. Otherwise, add the UID to the current-session attendance list, show “Attendance OK” with the student's name, switch on the green LED, beep once, and print the event to Serial Monitor.
8. Finish communication with the RFID card and wait for the next scan.

9. Software Implementation

Libraries and platform

The source is written for Arduino IDE and an ESP32 board package. It includes SPI.h, MFRC522.h, Wire.h, LiquidCrystal_I2C.h, and Preferences.h. Install the MFRC522 library and a compatible LiquidCrystal_I2C library before compiling.

Serial commands

Command	Purpose / example
HELP	Print the command list and usage notes.
LIST	List all registered card UIDs and names.
ADD <UID> <Name>	Register or update a student, for example: ADD 04:A1:B2:C3 Student One. Replace the UID with the exact value printed by the reader.

Use Serial Monitor at 115200 baud with line ending set to Newline. The UID example above is only a format example and is not a real card UID.

Storage behavior

The Preferences namespace is “attendance”. Registered student mappings are saved to non-volatile storage. Attendance de-duplication is held in a runtime array and resets after power cycle. The current serial event reports millis() as uptime_ms; this is elapsed time since boot, not real calendar time.

10. Testing and Results

Functional testing of the assembled prototype was completed successfully. The LCD displayed “Smart Attendance / Scan your card”. A registered card was recognized and the student name with attendance confirmation appeared on the LCD, with green LED and buzzer feedback. An unregistered card displayed “Card not found / Not registered” with red LED and error beeps. Re-scanning an already-marked card displayed “Already marked” and prevented a duplicate during the current power session. Card UIDs and attendance events were viewed in Serial Monitor.

Test / checkpoint	Test result	Observed result / notes
Physical assembly	Assembly photographed	See the prototype views in Section 11.
Arduino IDE compilation	Successful	Firmware compiled successfully with the required ESP32, RFID and LCD libraries.
Upload to ESP32	Successful	Firmware uploaded to the ESP32 used for the final test.
LCD live text	Successful	LCD startup prompt and status messages were verified.
Registered RFID attendance scan	Successful	Registered UID recognized; student name and attendance confirmation displayed. UID and event visible in Serial Monitor.
Unregistered / duplicate card behavior	Successful	Unknown card: not-registered message, red LED and error beeps. Repeat scan: “Already marked”; duplicate prevented for the session.
Calendar date/time attendance log	Not implemented	Elapsed uptime only; calendar time requires an RTC or synchronized time source.

All functional tests within the implemented project scope passed successfully. Calendar date/time capture and persistent attendance-event history are not included in this version and are possible future enhancements.

11. Project Photos and Evidence

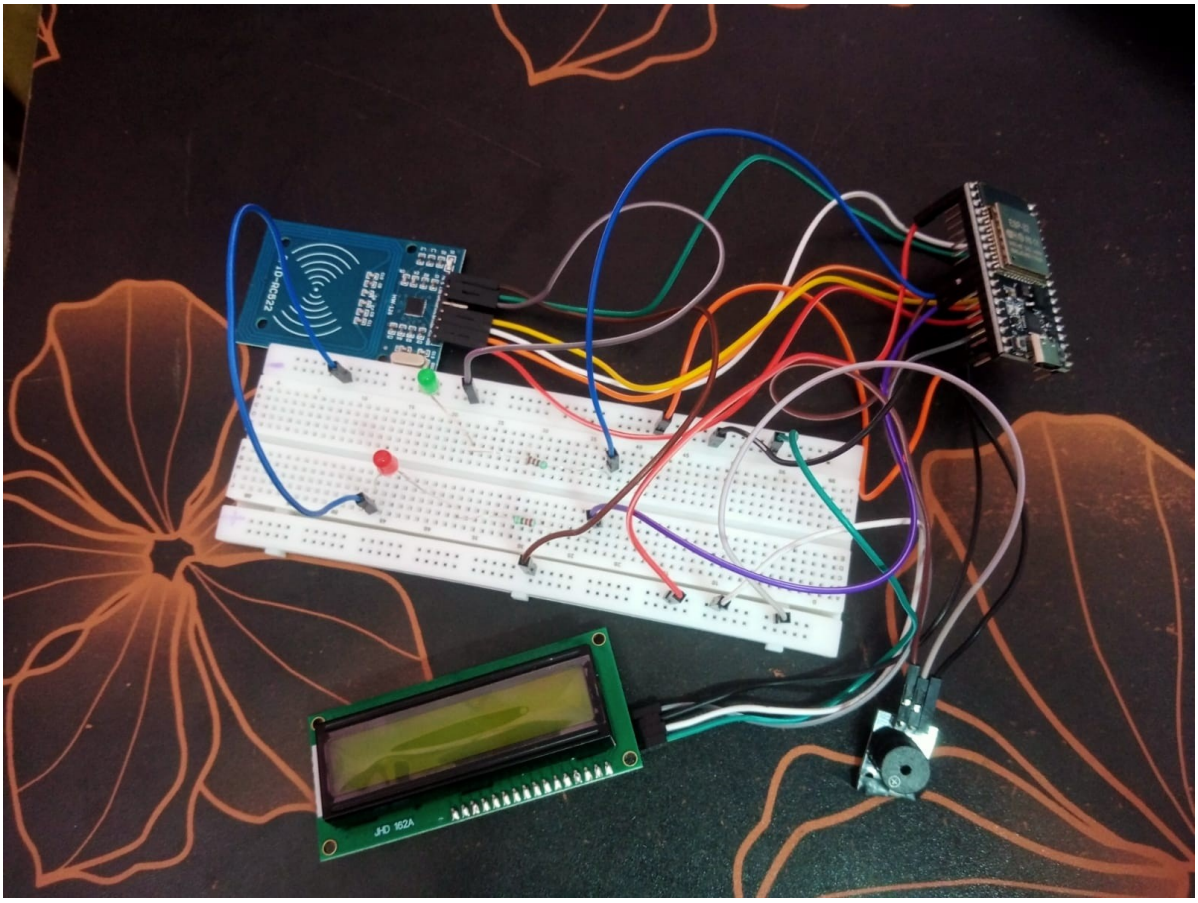


Figure 1. Prototype assembly - view 1 showing ESP32, RC522, LCD, indicators and buzzer.

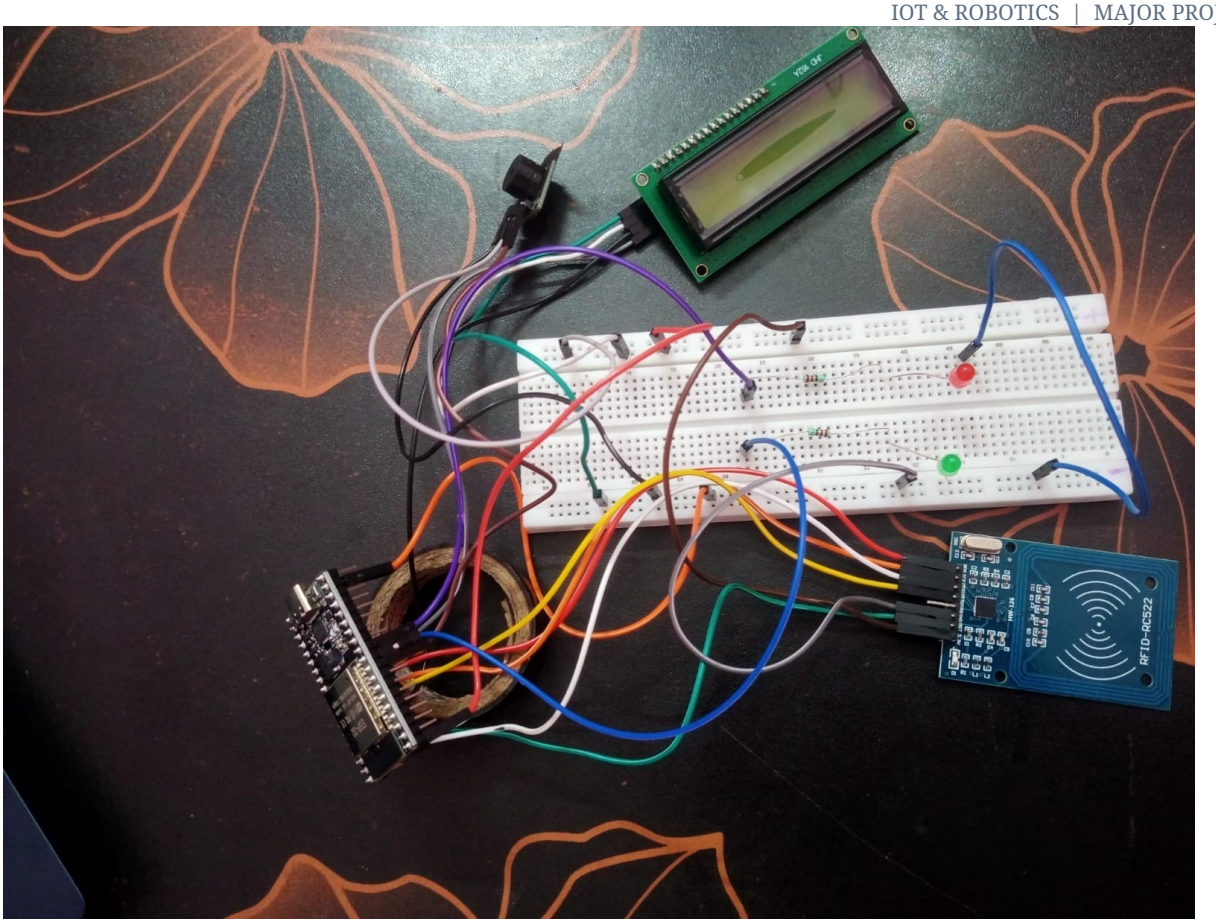


Figure 2. Prototype assembly - alternate view showing connections among the modules.

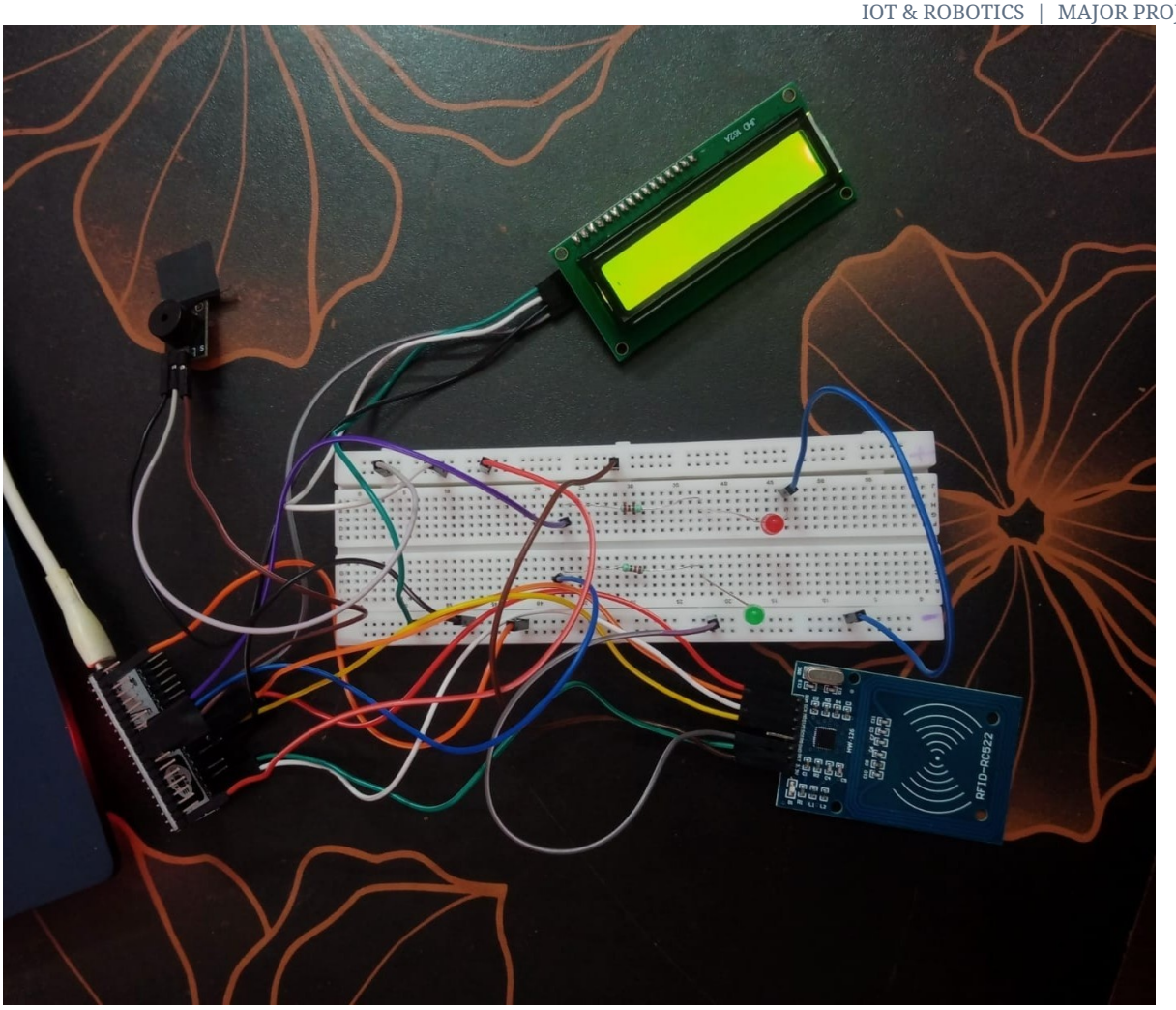


Figure 3. Additional view of the assembled hardware prototype.

The photographs show the assembled prototype and its connected modules.

12. Limitations and Future Enhancements

- Add an RTC module or reliable network time to record calendar date and time.
- Write attendance events persistently to an SD card, database, or authenticated server.
- Add a controlled export/reporting process for attendance logs.
- Improve identity assurance and anti-proxy procedures; a card UID alone is not proof that the cardholder is the registered student.
- Improve cable management and add a protective enclosure for a more durable installation.
- Add a tested reset/session policy, administrative authentication, backups, and protection for student data.

13. Conclusion

The Smart Attendance System combines an ESP32, RC522 RFID reader, 16x2 I2C LCD, LEDs, and a buzzer to provide an automated attendance workflow. The system supports student UID

registration, name lookup, LCD and indicator feedback, duplicate protection during a power session, and Serial Monitor commands. The prototype was assembled and tested, demonstrating the practical use of RFID technology for attendance management. Future improvements may include real-time date/time and persistent attendance-log export.

Appendix A - Complete Arduino Source Code

The full commented source below is also provided as the separate Smart_Attendance_ESP32_RFID.ino file for direct use in Arduino IDE.

```
/*
  Smart Attendance System using ESP32 + RC522 RFID + I2C 16x2 LCD

  Wiring assumed:
    RC522: SDA/SS=GPIO5, SCK=18, MOSI=23, MISO=19, RST=22, 3.3V, GND
    I2C LCD: SDA=GPIO21, SCL=GPIO17, GND, VCC (use only a safe voltage/signal setup)
    Buzzer module: S=GPIO25, middle/VCC=3V3 only if its rating allows, -=GND
    Green LED: GPIO26 -> 220 ohm resistor -> LED long leg; short leg -> GND
    Red LED:   GPIO27 -> 220 ohm resistor -> LED long leg; short leg -> GND

  Libraries to install in Arduino IDE:
    - MFRC522
    - LiquidCrystal I2C (for LiquidCrystal_I2C.h)

  Serial Monitor: 115200 baud, line ending = Newline

  Serial commands:
    HELP
    LIST
    ADD AA:BB:CC:DD Student Name   (replace with the real UID printed by your reader)

  Student registrations are stored in ESP32 Preferences and survive reboot.
  Attendance marks are kept for the current power session and printed to Serial.
  Without an RTC or Wi-Fi time, uptime_ms is elapsed time since boot, not date/time.
*/

#include <SPI.h>
#include <MFRC522.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Preferences.h>

// ---------- Pin configuration ----------
const int RFID_SS_PIN = 5;
const int RFID_RST_PIN = 22;
const int RFID_SCK_PIN = 18;
const int RFID_MISO_PIN = 19;
const int RFID_MOSI_PIN = 23;

const int LCD_SDA_PIN = 21;
const int LCD_SCL_PIN = 17;
const uint8_t LCD_I2C_ADDRESS = 0x27;  // If the LCD stays blank, it may use 0x3F instead.

const int BUZZER_PIN = 25;
const int GREEN_LED_PIN = 26;
const int RED_LED_PIN = 27;

// Most 16x2 I2C LCDs are 16 columns x 2 rows.
LiquidCrystal_I2C lcd(LCD_I2C_ADDRESS, 16, 2);
MFRC522 rfid(RFID_SS_PIN, RFID_RST_PIN);
Preferences preferences;

// ---------- Student storage ----------
const int MAX_STUDENTS = 30;
struct Student {
  String uid;
  String name;
};
Student students[MAX_STUDENTS];
int studentCount = 0;
```

```
// Attendance is once per registered card per power session.
String attendedUIDs[MAX_STUDENTS];
int attendedCount = 0;


bool messageIsShowing = false;
unsigned long messageUntil = 0;
const unsigned long MESSAGE_DURATION_MS = 2200;


// Turn any UID spelling into a consistent format, e.g. AA:BB:CC:DD.
String normalizeUID(String input) {
  input.trim();
  String hexOnly = "";
  for (unsigned int i = 0; i < input.length(); i++) {
    char c = input.charAt(i);
    if (c >= 'a' && c <= 'f') c = c - 'a' + 'A';
    if ((c >= '0' && c <= '9') || (c >= 'A' && c <= 'F')) {
      hexOnly += c;
    }
  }


  String formatted = "";
  for (unsigned int i = 0; i < hexOnly.length(); i++) {
    if (i > 0 && i % 2 == 0) formatted += ':';
    formatted += hexOnly.charAt(i);
  }
  return formatted;
}


bool isUsableUID(const String &uid) {
  if (uid.length() < 11) return false;  // At least 6 bytes in colon-separated format.
  int hexCount = 0;
  for (unsigned int i = 0; i < uid.length(); i++) {
    if (uid.charAt(i) != ':') hexCount++;
  }
  return (hexCount >= 8 && hexCount % 2 == 0);
}


String getScannedUID() {
  String uid = "";
  for (byte i = 0; i < rfid.uid.size; i++) {
    if (i > 0) uid += ':';
    if (rfid.uid.uidByte[i] < 0x10) uid += '0';
    uid += String(rfid.uid.uidByte[i], HEX);
  }
  uid.toUpperCase();
  return uid;
}


void showMessage(String line1, String line2, unsigned long durationMs = MESSAGE_DURATION_MS) {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print(line1.substring(0, 16));
  lcd.setCursor(0, 1);
  lcd.print(line2.substring(0, 16));
  messageIsShowing = true;
  messageUntil = millis() + durationMs;
}


void showHomeScreen() {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Smart Attendance");
  lcd.setCursor(0, 1);
  lcd.print("Scan your card");
  messageIsShowing = false;
}


void setIndicators(bool greenOn, bool redOn) {
  digitalWrite(GREEN_LED_PIN, greenOn ? HIGH : LOW);
  digitalWrite(RED_LED_PIN, redOn ? HIGH : LOW);
}
```

```
// Assumes the buzzer module is active-HIGH and supports 3.3V supply.
void beepOnce(unsigned int durationMs) {
  digitalWrite(BUZZER_PIN, HIGH);
  delay(durationMs);
  digitalWrite(BUZZER_PIN, LOW);
}


void beepSuccess() {
  beepOnce(120);
}


void beepError() {
  beepOnce(90);
  delay(100);
  beepOnce(90);
}


int findStudentByUID(const String &uid) {
  for (int i = 0; i < studentCount; i++) {
    if (students[i].uid == uid) return i;
  }
  return -1;
}


bool alreadyAttended(const String &uid) {
  for (int i = 0; i < attendedCount; i++) {
    if (attendedUIDs[i] == uid) return true;
  }
  return false;
}


void loadStudents() {
  studentCount = preferences.getUInt("count", 0);
  if (studentCount > MAX_STUDENTS) studentCount = MAX_STUDENTS;


  for (int i = 0; i < studentCount; i++) {
    String uidKey = "u" + String(i);
    String nameKey = "n" + String(i);
    students[i].uid = normalizeUID(preferences.getString(uidKey.c_str(), ""));
    students[i].name = preferences.getString(nameKey.c_str(), "Student");
  }
}


void saveStudent(int index) {
  String uidKey = "u" + String(index);
  String nameKey = "n" + String(index);
  preferences.putString(uidKey.c_str(), students[index].uid);
  preferences.putString(nameKey.c_str(), students[index].name);
  preferences.putUInt("count", studentCount);
}


void printHelp() {
  Serial.println();
  Serial.println("--- Smart Attendance commands ---");
  Serial.println("HELP");
  Serial.println("LIST");
  Serial.println("ADD AA:BB:CC:DD Student Name");
  Serial.println("Use the exact UID printed when you scan your card.");
  Serial.println("Serial Monitor: 115200 baud, line ending Newline.");
  Serial.println();
}


void listStudents() {
  Serial.println("--- Registered cards ---");
  if (studentCount == 0) {
    Serial.println("No students registered yet. Scan a card, then use ADD <UID> <Name>.");
    return;
  }
  for (int i = 0; i < studentCount; i++) {
    Serial.print(i + 1);
    Serial.print(". ");
    Serial.print(students[i].name);
```

```
    Serial.print(" | UID: ");
    Serial.println(students[i].uid);
  }
}

void addOrUpdateStudent(String uidInput, String name) {
  String uid = normalizeUID(uidInput);
  name.trim();

  if (!isUsableUID(uid) || name.length() == 0) {
    Serial.println("Format: ADD AA:BB:CC:DD Student Name");
    Serial.println("Use the UID printed by the RFID reader.");
    return;
  }

  int existingIndex = findStudentByUID(uid);
  if (existingIndex >= 0) {
    students[existingIndex].name = name;
    saveStudent(existingIndex);
    Serial.print("Updated registered student: ");
    Serial.print(name);
    Serial.print(" | UID: ");
    Serial.println(uid);
    return;
  }

  if (studentCount >= MAX_STUDENTS) {
    Serial.println("Registration full: maximum of 30 students reached.");
    return;
  }

  students[studentCount].uid = uid;
  students[studentCount].name = name;
  saveStudent(studentCount);
  studentCount++;
  preferences.putUInt("count", studentCount);

  Serial.print("Registered: ");
  Serial.print(name);
  Serial.print(" | UID: ");
  Serial.println(uid);
}

void processSerialCommand() {
  if (!Serial.available()) return;

  String line = Serial.readStringUntil('\n');
  line.trim();
  if (line.length() == 0) return;

  int firstSpace = line.indexOf(' ');
  String command = (firstSpace < 0) ? line : line.substring(0, firstSpace);
  command.toUpperCase();

  if (command == "HELP") {
    printHelp();
  } else if (command == "LIST") {
    listStudents();
  } else if (command == "ADD") {
    if (firstSpace < 0) {
      Serial.println("Format: ADD AA:BB:CC:DD Student Name");
      return;
    }
    String rest = line.substring(firstSpace + 1);
    rest.trim();
    int separator = rest.indexOf(' ');
    if (separator < 0) {
      Serial.println("Please include both the card UID and student name.");
      Serial.println("Example: ADD AA:BB:CC:DD Student One");
      return;
    }
    String uidText = rest.substring(0, separator);
```

```
    String studentName = rest.substring(separator + 1);
    studentName.trim();
    addOrUpdateStudent(uidText, studentName);
  } else {
    Serial.println("Unknown command. Type HELP.");
  }
}

void handleCard(String uid) {
  Serial.print("Card UID: ");
  Serial.println(uid);

  int index = findStudentByUID(uid);
  if (index < 0) {
    setIndicators(false, true);
    showMessage("Card not found", "Not registered");
    beepError();
    Serial.println("This card is not registered.");
    Serial.print("To register it, type: ADD ");
    Serial.print(uid);
    Serial.println(" Student Name");
    return;
  }

  const String studentName = students[index].name;
  if (alreadyAttended(uid)) {
    setIndicators(true, false);
    showMessage("Already marked", studentName);
    beepOnce(70);
    Serial.print("DUPLICATE, name=");
    Serial.print(studentName);
    Serial.print(", UID=");
    Serial.println(uid);
    return;
  }

  if (attendedCount < MAX_STUDENTS) {
    attendedUIDs[attendedCount++] = uid;
  }
  setIndicators(true, false);
  showMessage("Attendance OK", studentName);
  beepSuccess();

  Serial.print("ATTENDANCE, name=");
  Serial.print(studentName);
  Serial.print(", UID=");
  Serial.print(uid);
  Serial.print(", uptime_ms=");
  Serial.println(millis());
  Serial.println("Note: uptime_ms is elapsed time since boot, not calendar date/time.");
}

void setup() {
  Serial.begin(115200);
  Serial.setTimeout(100);

  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(GREEN_LED_PIN, OUTPUT);
  pinMode(RED_LED_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);
  setIndicators(false, false);

  Wire.begin(LCD_SDA_PIN, LCD_SCL_PIN);
  lcd.init();
  lcd.backlight();
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Smart Attendance");
  lcd.setCursor(0, 1);
  lcd.print("Starting...");

  // Explicit SPI pins for the ESP32.
```

```
  SPI.begin(RFID_SCK_PIN, RFID_MISO_PIN, RFID_MOSI_PIN, RFID_SS_PIN);
  rfid.PCD_Init();
  delay(50);

  preferences.begin("attendance", false);
  loadStudents();

  Serial.println();
  Serial.println("Smart Attendance System ready.");
  Serial.print("Registered students: ");
  Serial.println(studentCount);
  printHelp();

  showHomeScreen();
}

void loop() {
  processSerialCommand();

  if (messageIsShowing) {
    if ((long)(millis() - messageUntil) >= 0) {
      setIndicators(false, false);
      showHomeScreen();
    } else {
      return;
    }
  }

  if (!rfid.PICC_IsNewCardPresent()) return;
  if (!rfid.PICC_ReadCardSerial()) return;

  String uid = getScannedUID();
  handleCard(uid);

  // Finish communication with this card.
  rfid.PICC_HaltA();
  rfid.PCD_StopCrypto1();
}
```